Document library & viewer: research options (pre-mock)

> **6 October 2026.** Research for a Rocket-side **library** (saved plans, browser downloads, chat attachments, workspace docs) plus a **viewer** where users and agents can open, read, highlight, and edit without hitting implementation walls. This note compares proven stacks; mockups come next.

What Rocket has today

Surface	Behavior	Agent today
Artifact panel (`ArtifactPanel.tsx`)	Markdown rendered via `ChatMarkdown`; other files as source with line highlight; PDF attachments as **raster page images**; turn diffs	Models edit files on disk (`Write` / patch tools) and open paths with `npm run open`; panel does not expose a rich edit or annotate API
Attachments (phases 1–2 shipped)	Host-owned copies, SQLite metadata, side-panel preview	Same as above; preview ≠ provider delivery
Built-in browser	Full web pages, downloads to workspace	`browser_*` tools
design-library/	UI reference templates for goals — **not** a document vault	N/A

There is no unified "library" browse/search across downloads + plans + attachments yet (`attachments-roadmap.md` mentions "Files in this chat" only if menus prove insufficient).

Requirements (from product + agent use)

1. **Proven viewers** — battle-tested rendering for PDF, markdown/plans, plain text/code, images; Office formats only if we explicitly scope them.
2. **Agent-capable** — programmatic open, read structure/text, navigate (page / heading / line), highlight, annotate, and edit where appropriate, mappable to Rocket MCP + `rocket_ui_*` map.
3. **No fake affordances** — if the UI shows a highlight tool, agents must be able to apply the same change without pixel-hacking or undocumented internals.
4. **Disk as source of truth** — align with attachments: host stores originals; viewer watches files and refreshes when agents write bytes (agents already excel at file tools).
5. **Electron embed** — runs in the renderer beside chat; no requirement to ship a second app shell.
6. **Security** — uploaded HTML and scripts stay sandboxed; previews are not privileged app pages (`attachments-roadmap.md`).

How agents should control the viewer (target architecture)

Two complementary control planes — both should exist:

Plane	Mechanism	Best for
File	Host file APIs + existing agent write/read tools; `fs.watch` → viewer reload	Markdown plans, code, text exports; bulk edits
Viewer SDK	Host MCP tools → renderer dispatcher → viewer instance (`open` , `goto` , `highlight` , `annotate` , `getSelection`)	PDF highlights, ink, comments; WYSIWYG marks without rewriting PDF bytes; scroll-to-line in large PDFs

The library UI is mostly Rocket product code (index, filters, provenance: download vs attachment vs plan). The **viewer engine** is what this research selects.

Evaluation criteria

Criterion	Question
Programmatic API	Documented create/read/update for annotations, selection, navigation?
Format breadth	PDF, MD, code, images, Office?
Persistence	Save highlights into file (PDF annotations, sidecar JSON) or export?
License / ship cost	OK for commercial Electron product?
Bundle weight	Acceptable in Electron + Vite?
Map + MCP fit	Can every user action be named in `app-map` and invoked by tools?

Option A — Composed “best of breed” stack (recommended baseline for mock)

Use one **library shell** in Rocket; pick a **specialist engine per format** with a real SDK.

Format	Library	Agent / programmatic surface	License
Markdown & plans	**CodeMirror 6** for source; optional **Tiptap** for rendered + highlight mode	CM6: immutable doc + `view.dispatch(transaction)` with `{ changes, selection }` (System Guide). Tiptap: `setContent` / `insertContentAt` / `setHighlight({ color })` on JSON or HTML (setContent, Highlight)	MIT
Code / text / logs	**CodeMirror 6** + language packs	Same transaction API; line-accurate `goto` via selection	MIT
PDF	**Nutrient Web SDK** *or* **Apryse WebViewer** (see B/C) — not raw PDF.js alone for annotate	Nutrient: `instance.create(annotation)` , `getAnnotations` , `ensureChangesSaved` (create annotations). Apryse: `AnnotationManager.addAnnotation` , import XFDF (create annotations)	Commercial
Images	Native `<img>` + zoom; **pen/highlights** via canvas layer or licensed **tldraw**	tldraw: `editor.createShape` , bulk ops, transactions (Editor) — **production requires license key** (tldraw license)	Image: built-in; tldraw: commercial / hobby
Office (optional phase)	**ONLYOFFICE Docs** embed *or* defer	`AnnotateParagraph` , `SelectAnnotationRange` , `callCommand` + Office API (annotations guide) — needs Document Server	Server AGPL + commercial embed terms

PDF.js alone: Mozilla documents viewing plus a **subset** of annotation types in the default viewer (PDF.js FAQ), but there is no first-class, stable public API comparable to Nutrient/Apryse for building agent tools. Rocket today already uses **pre-rendered page images** for PDF preview — fine for reading, poor for highlights and text-linked navigation.

Pros: Matches how agents already work (file edits + SDK for overlays); each piece is industry standard; MIT stack for text/md; map each tool to one SDK call.

Cons: Multiple integrations; PDF vendor cost; Office needs a server or a conversion story.

Agent blockers: Minimal if MCP tools wrap SDK + file watch; main gap is **choosing PDF vendor** and **annotation persistence format** (Instant JSON / XFDF / embedded).

Option B — Single vendor: Apryse WebViewer

One embed for **PDF + Office** (via conversion) with shared **AnnotationManager** and optional **PDFNet** full API for low-level ops (annotation guide, AnnotationManager).

Strengths	Broad format story; mature programmatic annotations; used in legal/enterprise viewers
Agent fit	Strong — create/update/delete annotations, events on `documentLoaded` / `annotationsLoaded`
Weaknesses	Commercial license; bundle size; Office fidelity depends on conversion pipeline
vs Nutrient	Overlapping PDF annotation APIs; both are the "pay for no blockers" PDF path

When to prefer: PDF + Word/Excel preview/edit in one pane matters more than minimal deps.

Option C — Single vendor: Nutrient Web SDK (formerly PSPDFKit for Web)

PDF-first (also images, Office via conversion in their ecosystem). Documented JS annotation constructors, Instant JSON / XFDF export, `instance.create` / `update` / `delete` (annotations overview).

Strengths	Excellent PDF annotation UX + API; framework-agnostic
Agent fit	Strong for PDF highlight, ink, notes, links
Weaknesses	Commercial; less natural for editing `docs/*.md` plans in place

When to prefer: Library is **PDF-heavy** (downloads, reports) and markdown stays file/editor-based.

Option D — ONLYOFFICE Document Server + embed

Iframe `DocEditor` / DocSpace `initEditor` with `executeInEditor` for inserts (DocEditor, editor mode). **9.2+** text annotation API aimed at AI assistants (`AnnotateParagraph`, etc.).

Strengths	Real Word/Excel/PPT editing; AI-oriented annotation API

Agent fit	Good **inside** the editor iframe via plugin/command bridge; awkward for plain `.md` plans
Weaknesses	Requires running Document Server (ops); paid features for some APIs; not a single library for images/PDF/markdown

When to prefer: Users expect **Office-native** edit + review, not just PDF/markdown.

Option E — Embed or fork an "agent-ready document app"

Project	What it is	Agent story	Fit for Rocket embed
Doc77	Local doc portal + Electron; PDF/MD/Office/code; **MCP server** (stdio + HTTP) with approval queue on writes	Agents drive filesystem via MCP while user watches	**Reference architecture**, not drop-in: second product shell, duplicate MCP, approval model differs from Rocket's turn-based agent
DocuLight	Electron markdown viewer + MCP (`open_markdown` , `update_markdown` , search)	Strong for **MD-only** agent loops	Too narrow; markdown-only
HyperRead	Electron reader MD/PDF/EPUB + plugin API	Plugin events; **no** documented agent MCP	**AGPL**; fork friction
AgenticPDF	TS PDF parse/extract + tool schemas	Agent **ingestion**, not a human viewer	Complement for RAG, not the panel UI (**AGPL**)

Pros: Doc77 validates "MCP + preview + multi-format" UX.

Cons: Embedding means maintaining a fork or iframe a separate server; Rocket already owns shell, store, and MCP — importing another MCP server confuses the single host model.

When to prefer: Time-to-demo for **breadth** only; long term, steal patterns (index, vendor bundling, MCP tool names), not the app.

Option F — Incremental: extend current artifact panel only

Keep raster PDF pages + `<pre>` text + `ChatMarkdown` ; add library list in SQLite.

Pros	Smallest diff
Cons	**Fails** the "no realistic blockers" bar for PDF highlight, ink, comments, Office, and structured navigation
Agent fit	File tools only; no viewer SDK

When to prefer: Never as the end state; only as phase 0 while picking B vs C.

Comparison matrix (short)

Option	MD/plans edit	PDF annotate	Images mark up	Office	Agent API depth	Ship cost
A Composed	Excellent (CM6/Tiptap)	Excellent *with B or C*	Good (canvas / tldraw)	Optional (D)	High	PDF vendor + optional tldraw
B Apryse	Needs CM6/Tiptap alongside	Excellent	PDF-centric	Good	High	$$
C Nutrient	Needs CM6/Tiptap alongside	Excellent	Viewer supports images	Convert	High (PDF)	$$
D ONLYOFFICE	Weak	Via conversion	No	Excellent	Medium–high (iframe)	Server + license
E Doc77-style	Good	Good	Good	Good	MCP (external)	Maint fork
F Incremental	Read + file tools	Poor	Poor	Poor	Low	$

Recommended direction before mock

1. **Product shape:** One **Library** side panel (or rail section): unified index of `{ path, kind, source: download | attachment | workspace, chatId?, modifiedAt }` — Rocket code, not a third-party app.
2. **Viewer engines (mock with these):**
   - **Text / MD / code / plans:** CodeMirror 6 (primary); toggle “Preview” with Tiptap + Highlight for user-style marks agents can mirror with `setHighlight` / transactions.
   - **PDF:** Pick **Nutrient vs Apryse** in mock (same UX: thumbnails, page goto, highlight, note); expose tools `document.open`, `document.gotoPage`, `document.highlightText|Rect`, `document.listAnnotations`.
   - **Images:** Zoom + crop (deterministic host tools per user preference); pen layer in a later phase — evaluate **tldraw** vs simpler canvas after PDF choice (license vs build-your-own).
3. **Office: Out of v1 mock** unless you explicitly want Docx in scope — then ONLYOFFICE or Apryse conversion, not LibreOffice-in-renderer.
4. **Do not embed Doc77** as the shell; treat it as a **UX/MCP reference** only.
5. **Agent principle:** Any highlight visible to the user must be creatable via (a) file edit for text/md, or (b) documented SDK call for PDF/image overlays, wired through the same dispatcher as `rocket_ui_act`.

Decisions (locked for mock)

These choices follow Rocket’s existing contract: host-owned bytes, agents strong on file tools, viewer SDK for what files cannot express cleanly (PDF marks), and no second MCP server inside the app.

#	Decision	Choice	Rationale
1	**PDF engine**	**Nutrient Web SDK**	PDF is the hard part (highlights, ink, notes, page goto). Nutrient's instance API (`create` , `getAnnotations` , export) matches agent tools without Apryse's Core/PDFNet split. Office is out of v1, so Apryse's breadth does not pay for itself yet. **Spike:** 2-day Electron trial embed before mock is frozen in HTML.
2	**Markdown / plans**	**CodeMirror 6 source-first**, optional **read-only preview tab** (reuse `ChatMarkdown`)	Plans stay plain `.md` on disk; agents already patch files. Tiptap v1 would fight round-trips and duplicate schema. Programmatic highlights use CM6 decorations + stable `{ path, line, from, to }` anchors in a sidecar. WYSIWYG edit (Tiptap) only if user research demands it later.
3	**Annotation persistence**	**Host sidecar per library entry**, not silent in-place PDF rewrite	Original download paths and attachment "original retained" semantics stay intact. Store Nutrient **Instant JSON** (or equivalent) under the host attachment/library store keyed by `libraryEntryId` . User action **Save annotated copy** exports an embedded PDF when they want a portable file.
4	**Library scope v1**	**Unified index:** chat attachments + browser downloads into workspace + workspace docs user opens from chat (`npm run open` , plan paths)	Attachments already have IDs and SQLite metadata; downloads and plans without a unified shelf was the gap. **Not** a global filesystem crawler — scope is Rocket workspace roots + attachment store.
5	**Office**	**Library card + open externally** (Reveal / OS default app); **no** Document Server in v1	ONLYOFFICE is ops-heavy for a desktop agent shell. v1 honesty: show metadata and hand off to native apps. Revisit read-only in-viewer conversion (Nutrient or Apryse) only after PDF+md library ships.

Mock implications

- Shell: **Library** opens as a side panel (rail icon): **filter rail + thumbnail grid** first (DAM-style, per document-library.html); **card → viewer** with back to grid.
- Viewer: CM6 source + read-only md preview; Nutrient chrome on PDF; Office → external app.
- Agent tools (names tentative): `library.list` , `library.open` , `document.goto` , `document.highlight` , `document.annotate` , `document.getText` — all via host → renderer dispatcher, same pattern as `rocket_ui_act` .
- Image markup stays **out of v1** (attachments roadmap phase 4); images **zoom-only** in library.

Implementation quality bar (mock ≠ ship)

The HTML mock validates **layout and flow only**. Production must not inherit its placeholder visuals.

Area	Mock today	Ship bar
Thumbnails	CSS gradients and fake "pages"	Host-generated previews on import/index (reuse attachment preview pipeline): PDF page 1, image resize, md/code first screen, type fallback glyph; **preserve aspect ratio** (portrait stays portrait).
Grid	Fixed 5 columns, 15 static cards	Virtualised grid or paginated fetch from SQLite index; stable sort; skeleton while preview generates.
Filters	Non-functional `<select>`	Query indexed fields (`source` , `mime` , `chatId` , `modifiedAt`); debounced search; clear resets; filter state in URL or panel state for agent `library.list` parity.
Typography & density	Approximate Ranch spacing	Match Rocket `styles.css` / artifact panel rhythm; keyboard focus, narrow pane, hi-DPI thumbs.

Area	Mock today	Ship bar
Open / back	Instant swap	Preserve scroll position in grid; viewer loads from host path; `fs.watch` refresh; loading and error surfaces like attachments.
Viewer	Fake CM6 / PDF	Real CM6 + Nutrient trial embed; sidecar load/save; same actions exposed to `rocket_ui_act`.
Performance	All assets in one HTML file	Thumbnail cache on disk; cap concurrent generation; never block UI thread on large PDFs.
Tests	None	`app-map` place + act smoke; UI eval: grid → open md → back (fixture files).

Mock status (Oct 2026): direction approved (grid + filters + drill-in); **visual and data fidelity explicitly out of scope** until implementation slice 1 (index + real thumbnails).

Sources consulted

- Rocket: `src/renderer/src/components/ArtifactPanel.tsx`, `docs/attachments-roadmap.md`
- CodeMirror 6 System Guide — transactions / `dispatch`
- Tiptap setContent, Highlight extension
- PDF.js Getting Started, PDF.js FAQ (annotations)
- Nutrient Web — create annotations
- Apryse — create annotations, AnnotationManager API
- ONLYOFFICE — text annotations guide, DocEditor
- tldraw Editor / license, tldraw license
- Doc77 README (raw) — describes MCP + multi-format; states MIT license in README footer
- DocuLight MCP tools (markdown-focused; not re-fetched this session)

Next step

- **Mock (direction OK, quality TBD in code):** document-library.html — grid + filters, open plan/PDF/office, agent highlight.
- **Then:** implementation plan slice (index + thumbnails + grid UI) → `app-map` (`library`, `library.document { id }`) → Nutrient spike → viewer tools.